Implementation

Cognix – Personal Knowledge Operating System with AI Memory

1. Implementation Overview

Cognix is implemented as a set of independently deployable microservices that together form the memory substrate described in the project abstract: an ingestion & indexing pipeline, a structured memory store (vector index + concept graph + relational facts), an adaptive context-retrieval/compaction layer, and an orchestration layer that mediates every call to the underlying LLM. The system follows Service-Oriented principles – each capability is exposed behind a well-defined REST/async API, services communicate over lightweight message queues rather than direct calls, and the LLM orchestration layer is built as a LangGraph state machine so that retrieval, ranking, compaction and generation are explicit, inspectable nodes rather than one monolithic prompt.

The implementation is split into four logical layers: (1) a Next.js client application, (2) an API Gateway that authenticates requests and routes them to backend services, (3) a set of Python microservices (ingestion, embedding/indexing, graph, retrieval-compaction, orchestration) built on FastAPI, and (4) the storage layer (vector database, graph database, relational database, object storage, and a message broker for asynchronous ingestion jobs).

2. Technology Stack

Layer	Technology / Tool	Purpose
Frontend	Next.js (React), TypeScript, Tailwind CSS	Chat UI, memory dashboard, source/provenance viewer, admin console
API Gateway	FastAPI + Nginx / Traefik reverse proxy	Single entry point, auth (JWT), request routing to microservices, rate limiting
Orchestration	Python, LangGraph, LangChain	Stateful multi-node agent graph: query understanding → retrieval → compaction → generation → memory update
Ingestion Service	FastAPI, Celery, Redis (broker), unstructured.io / custom parsers	Async load, clean/normalize, chunk, and route raw chats/notes/documents
Embedding & Indexing	sentence-transformers / OpenAI-compatible embeddings, Qdrant (vector DB)	Generates embeddings, maintains the semantic search index (component 3.1)
Graph RAG Service	Neo4j (or NetworkX for the prototype), spaCy / LLM-based entity extraction	Builds and traverses the concept relationship graph (component 3.2)
Structured Memory	PostgreSQL, SQLAlchemy	Stores facts, entities, events, attributes and source/provenance (component 3.3)
LLM Layer	Self-hosted Qwen / Llama on Modal, Anthropic Claude & OpenAI GPT-4 APIs	Pluggable underlying LLM for generation and entity/relationship extraction
Infra & DevOps	Docker, Docker Compose, GitHub Actions, Modal	Containerized services, CI/CD pipeline, GPU deployment for self-hosted models
Observability	Prometheus + Grafana, structured logging	Token-usage tracking, retrieval latency, cache hit-rate dashboards

3. System Architecture

The diagram below (as defined in the project abstract) is used directly as the blueprint for the implementation. Each numbered block maps one-to-one onto a microservice or a well-defined module inside a service, which keeps the deployed system traceable back to the original design.

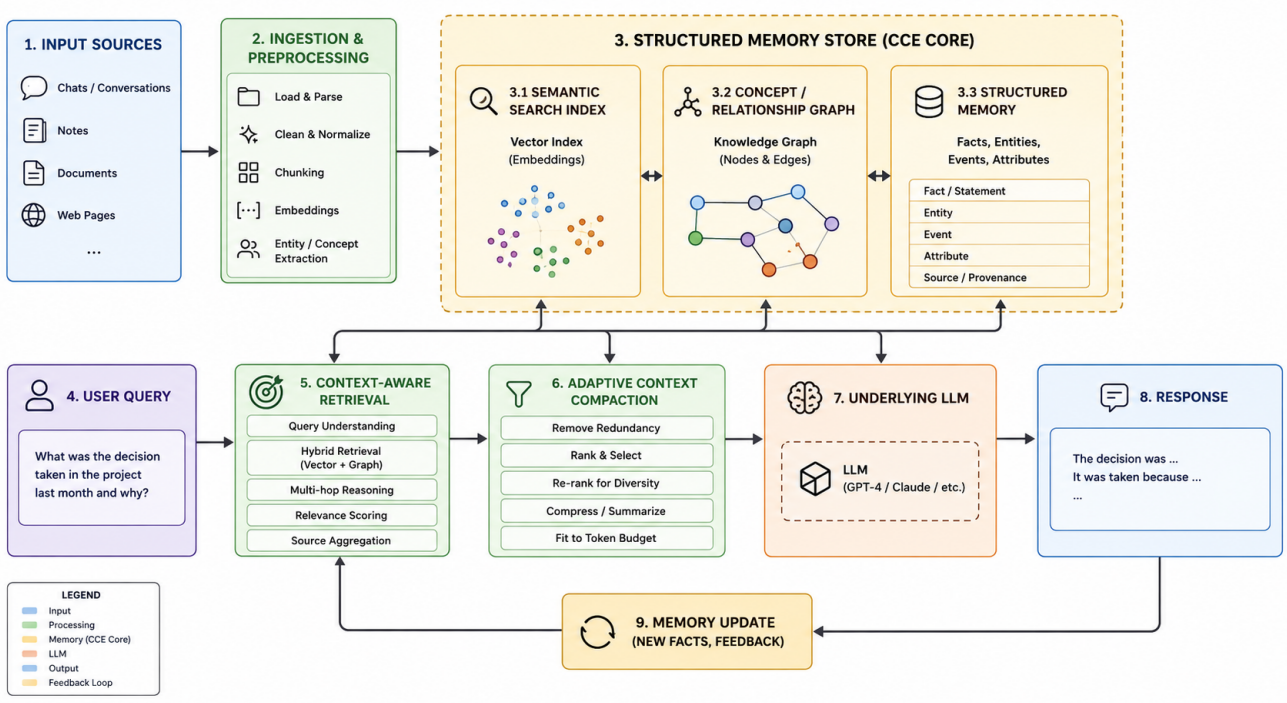


Fig. 1 – Cognix end-to-end architecture (input → ingestion → structured memory store → retrieval & compaction → LLM → response → memory update).

3.1 Service-to-Component Mapping

Diagram Block	Implementing Microservice
1–2. Input Sources / Ingestion & Preprocessing	ingestion-service (FastAPI + Celery workers, Redis queue)
3.1 Semantic Search Index	embedding-service (embedding model) + Qdrant vector store
3.2 Concept / Relationship Graph	graph-service (entity/relation extraction + Neo4j)
3.3 Structured Memory (facts, entities, events)	memory-store-service (PostgreSQL via SQLAlchemy)
4–6. User Query, Context-Aware Retrieval, Adaptive Compaction	retrieval-service, orchestrated as LangGraph nodes
7. Underlying LLM	llm-gateway-service (routes to self-hosted Qwen on Modal or hosted APIs)
8. Response	Streamed back through the API Gateway to the Next.js client (SSE)
9. Memory Update (feedback loop)	memory-update-service, triggered asynchronously after each turn

4. End-to-End Pipeline

4.1 Ingestion & Indexing Pipeline

- Raw chats, notes, documents and web pages are pushed onto a Redis-backed Celery queue by the ingestion-service so that large uploads never block the request/response cycle.
- Workers load and parse each source, clean and normalize text, and split it into semantically coherent chunks (paragraph- or turn-level, capped by token count).
- Each chunk is passed through an embedding model; the resulting vectors are written to Qdrant, and an entity/concept-extraction step (spaCy pipeline, backstopped by an LLM call for ambiguous cases) pulls out named entities, dates, and stated facts.
- Every stored unit keeps a source/provenance pointer back to the original chat turn or document, satisfying the structured-memory-unit requirement instead of storing flat text blocks.

4.2 Structured Memory Store (CCE Core)

- **Semantic Search Index** – a Qdrant collection indexed by embedding, filterable by user, topic, and recency; supports hybrid (dense + keyword) search.
- **Concept / Relationship Graph** – a Neo4j graph (NetworkX for the local/dev build) where nodes are entities/topics/events and edges capture relationships extracted during ingestion; enables multi-hop traversal instead of returning isolated chunks.
- **Structured Memory** – a PostgreSQL schema with tables for Fact/Statement, Entity, Event, Attribute, and Source/Provenance, giving the system a queryable ground truth alongside the vector and graph indexes.

4.3 Context-Aware Retrieval & Adaptive Compaction

This stage is implemented as a LangGraph graph rather than a single retrieval function, so each step is a separately testable node with its own state:

- **Query Understanding node** – classifies intent and extracts entities/time constraints from the user query.
- **Hybrid Retrieval node** – issues a vector-similarity search against Qdrant and a graph traversal (multi-hop) against Neo4j in parallel, then merges the candidate sets.
- **Multi-hop Reasoning node** – follows graph edges from the top vector hits to pull in related facts that a single vector search would miss (the Graph RAG step).
- **Relevance Scoring / Re-rank node** – scores candidates for relevance and diversity (cross-encoder re-ranker) and removes redundant memories.
- **Compression / Fit-to-Budget node** – summarizes or truncates the surviving memories so the final context fits a fixed token budget before being handed to the LLM node.

4.4 Generation & Memory Update

- The **LLM node** in the same LangGraph graph sends the compacted context plus the user query to the configured underlying LLM (self-hosted Qwen on Modal for experimentation, or GPT-4/Claude for production-quality answers) and streams the response back to the client.
- A final **Memory Update node** extracts any new facts, corrections, or explicit feedback from the exchange and writes them back into the vector index, graph, and structured memory store, closing the feedback loop shown as block 9 in the architecture diagram.

5. Microservice & API Design

In line with the SOA/microservices scope of the project, every capability above is wrapped in its own FastAPI service with a documented OpenAPI contract, and services never share a database – each owns its own storage and exposes data to the others only through its API or through domain events on the Redis broker.

Endpoint	Method	Description
/api/v1/ingest	POST	Accepts a chat/note/document; enqueues an ingestion job and returns a job id
/api/v1/memory/search	GET	Hybrid vector + graph search over a user's memory for a given query
/api/v1/memory/graph/{entity}	GET	Returns the local subgraph (neighbours, relations) around an entity
/api/v1/chat	POST (SSE)	Runs the LangGraph orchestration graph end-to-end and streams the response
/api/v1/memory/facts	GET/POST	CRUD over structured facts/entities/events for transparency and correction

5.1 Frontend (Next.js)

- A chat interface that streams responses via Server-Sent Events from /api/v1/chat.
- A “memory inspector” page that renders the concept graph (using a graph-visualization library) and lets the user see exactly which memories/sources were used to answer a query – directly surfacing the source/provenance data from the structured memory store.
- Server-side API routes in Next.js act as a thin BFF (backend-for-frontend), forwarding authenticated requests to the API Gateway so browser clients never talk to internal services directly.

6. Deployment

- Every microservice ships as its own Docker image; docker-compose.yml wires together the gateway, orchestration service, ingestion workers, Qdrant, Neo4j, PostgreSQL and Redis for local development and grading demos.
- Self-hosted models (Qwen) are deployed on Modal for GPU access without maintaining dedicated hardware, while hosted APIs (GPT-4 / Claude) are used interchangeably behind the llm-gateway-service so the rest of the system is model-agnostic.
- GitHub Actions runs linting, unit tests per service, and builds/pushes container images on every merge, keeping the SOA boundaries enforced by CI rather than convention alone.

7. Representative Code Snippets

The snippets below are simplified excerpts that illustrate how each architectural component is actually wired up in code, using the stack listed in Section 2.

7.1 LangGraph Orchestration Graph

The retrieval → compaction → generation → memory-update pipeline (blocks 4–9 in Fig. 1) is defined as an explicit LangGraph state graph, so every step is inspectable and independently testable.

orchestrator/graph.py

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class CognixState(TypedDict):
    query: str
    user_id: str
    candidates: list
    context: str
    answer: str

graph = StateGraph(CognixState)
graph.add_node("understand_query", understand_query_node)
graph.add_node("hybrid_retrieve", hybrid_retrieve_node)
graph.add_node("multi_hop_reason", multi_hop_reason_node)
graph.add_node("rerank_and_compact", rerank_and_compact_node)
graph.add_node("generate", generate_node)
graph.add_node("update_memory", update_memory_node)

graph.set_entry_point("understand_query")
graph.add_edge("understand_query", "hybrid_retrieve")
graph.add_edge("hybrid_retrieve", "multi_hop_reason")
graph.add_edge("multi_hop_reason", "rerank_and_compact")
graph.add_edge("rerank_and_compact", "generate")
graph.add_edge("generate", "update_memory")
graph.add_edge("update_memory", END)

cognix_app = graph.compile()
```

7.2 FastAPI Ingestion Endpoint

The ingestion-service exposes a single async endpoint; the heavy parsing/embedding work is pushed onto a Celery queue so the request returns immediately with a job id.

services/ingestion/main.py

```
from fastapi import FastAPI, UploadFile
from .tasks import process_source

app = FastAPI(title="cognix-ingestion-service")

@app.post("/api/v1/ingest")
async def ingest_source(user_id: str, source_type: str, file: UploadFile):
    raw_bytes = await file.read()
    job = process_source.delay(
        user_id=user_id,
        source_type=source_type,   # chat | note | document | webpage
        payload=raw_bytes,
    )
    return {"job_id": job.id, "status": "queued"}
```

7.3 Hybrid Retrieval – Vector + Graph

The retrieval-service node queries the Qdrant vector index and the Neo4j concept graph in parallel, then merges the two candidate sets before scoring.

services/retrieval/hybrid.py

```
async def hybrid_retrieve(query_embedding, entities, user_id, k=20):
    vector_hits, graph_hits = await asyncio.gather(
        qdrant_client.search(
            collection_name=f"memory_{user_id}",
            query_vector=query_embedding,
            limit=k,
        ),
        graph_client.run(
            """MATCH (e:Entity)-[r*1..2]-(m:Memory)
               WHERE e.name IN $entities AND m.user_id = $user_id
               RETURN DISTINCT m, r LIMIT $k""",
            entities=entities, user_id=user_id, k=k,
        ),
    )
    return merge_and_dedupe(vector_hits, graph_hits)
```

7.4 Adaptive Context Compaction

The compaction node re-ranks the merged candidates, drops redundant memories, and greedily fills a fixed token budget instead of always returning a fixed top-K.

services/retrieval/compact.py

```
def compact_to_budget(candidates, query, token_budget=1500):
    ranked = cross_encoder_rerank(query, candidates)
    deduped = remove_redundant(ranked, similarity_threshold=0.92)

    selected, used_tokens = [], 0
    for memory in deduped:
        cost = count_tokens(memory.text)
        if used_tokens + cost > token_budget:
            memory.text = summarize(memory.text, max_tokens=token_budget - used_tokens)
            cost = count_tokens(memory.text)
            if used_tokens + cost > token_budget:
                break
        selected.append(memory)
        used_tokens += cost
    return selected
```

8. Evaluation Methodology

Cognix is evaluated against two baselines – plain conversation-history replay (dumping raw history into the prompt) and a standard chunk-based RAG pipeline (flat vector search, fixed top-K, no graph) – using the criteria defined in the abstract:

Metric	What it measures	How it is computed
Personalization accuracy	Whether responses correctly reflect the user's stated facts/preferences	Held-out Q&A pairs graded against ground-truth memory entries
Response relevance	Whether retrieved memories are actually pertinent to the query	Human/LLM-judged relevance scoring of retrieved context
Token efficiency	Context size needed to answer correctly	Prompt-token count vs. history-replay and fixed top-K RAG baselines
Retrieval latency	End-to-end time from query to compacted context	Measured via the observability stack (Prometheus)

9. Implementation Roadmap

- **Phase 1** – Ingestion service, embedding/indexing, and PostgreSQL structured memory (vector-only RAG baseline working end-to-end).
- **Phase 2** – Concept relationship graph and Graph RAG multi-hop retrieval added as a parallel retrieval path; hybrid merge logic in the LangGraph retrieval node.
- **Phase 3** – Adaptive context-compaction node (re-ranking, redundancy removal, token-budget fitting) and the memory-update feedback loop.
- **Phase 4** – Next.js frontend (chat + memory inspector), observability dashboards, and evaluation against the two baselines described above.

10. Conclusion

The implementation realizes Cognix as a set of loosely coupled, independently deployable microservices – ingestion, embedding/indexing, graph, structured memory, retrieval/compaction and LLM orchestration – connected through a FastAPI gateway and a Redis message broker, with a Next.js client on top. Using LangGraph to make the retrieval-compaction-generation pipeline an explicit, inspectable graph (rather than a single prompt) directly supports the project's goal of showing that structured, relationship-aware memory retrieval (Graph RAG) improves both the quality and token-efficiency of personalized AI interactions relative to conversation-history replay and standard RAG baselines.